



Universidade do Minho

Braga, Portugal

TRABALHO PRÁTICO - RELATÓRIO

COMPUTAÇÃO GRÁFICA

Fase I - Primitivas Gráficas

Grupo 37

Engenharia Informática 2025/26

Equipa de Trabalho:

A106936 - Duarte Escairo Brandão Reis Silva

A106932 - Luís António Peixoto Soares

A106856 - Tiago Silva Figueiredo

A104704 - Inês Ferreira Ribeiro

27 Fevereiro 2026

Índice

1. Introdução	1
2. Generator	1
3. Primitivas	2
3.1. Plano	2
3.2. Cubo	3
3.3. Esfera	4
3.4. Cone	6
3.5. Torus	8
3.6. Cilindro	9
4. Engine	11
4.1. Função	11
4.2. Teste externo	12
5. Conclusão	12
6. Bibliografia	13

1. Introdução

No âmbito da unidade curricular de Computação Gráfica, foi proposto o desenvolvimento de um projeto cujo objetivo consistiu na implementação de um mini motor gráfico 3D baseado numa estrutura de *scene graph*, o *engine*, e um gerador de modelos 3D, o *generator*, que forneça exemplos de utilização para serem utilizados pelo motor.

O *engine* é um programa que constrói cenas definidas em ficheiros XML, a partir dos modelos criados pelo *generator*, ou outro qualquer modelo desde que esse esteja no formato esperado, neste caso num ficheiro *.3d*.

Nesta primeira fase do projeto, o foco foi a implementação destes dois programas, e a geração dos modelos requeridos no enunciado, sendo esses modelos do plano, do cubo, do cone e da esfera. Como extras, o nosso gerador consegue também gerar modelos de torus e de cilindros.

2. Generator

O programa *generator* é o responsável pela criação de modelos geométricos 3D que serão posteriormente utilizados pelo *engine* para renderização. Este programa recebe como argumentos o tipo de primitiva geométrica a gerar, os seus parâmetros dimensionais (como raio, altura) e graus de subdivisão (grelha, *slices*, *stacks*), e o nome do ficheiro de saída onde os vértices serão armazenados. A arquitetura do *generator* foi desenvolvida de forma modular, onde cada primitiva possui uma função dedicada que calcula os vértices necessários para a sua representação através de triângulos, facilitando a extensibilidade e manutenção do código.

O processo de geração segue uma abordagem sistemática: primeiro, a função correspondente à primitiva escolhida calcula todos os vértices necessários e armazena-os num vetor de *floats*, onde cada conjunto de três valores consecutivos representa as coordenadas (*x*, *y*, *z*) de um vértice. De seguida, estes vértices são organizados em triângulos, seguindo sempre a regra da mão direita para garantir que as normais apontam para o exterior dos objetos. Por fim, a função *storeVertices* escreve estes dados no ficheiro de saída, colocando cada vértice numa linha separada, criando assim um ficheiro *.3d* pronto a ser lido pelo *engine*.

Tal como referido anteriormente, as primitivas requeridas no enunciado são o plano, o cubo, o cone e a esfera, contudo como trabalho extra, foram também desenvolvidas primitivas para o torus e cilindro. Desta forma, o programa *generator* pode ser usado das seguintes formas:

```
./generator plane      <length>      <divisions>      <dest file>
./generator box       <length>      <divisions>      <dest file>
./generator cone      <radius>      <height>        <slices> <stacks> <dest file>
./generator sphere    <radius>      <slices>        <stacks>      <dest file>
./generator torus     <majorRadius> <minorRadius> <slices> <stacks> <dest file>
./generator cylinder <radius>      <height>        <slices>      <dest file>
```

O formato *.3d* adotado para o armazenamento dos modelos foi escolhido pela sua simplicidade e eficácia. Cada linha do ficheiro contém exatamente três valores em vírgula flutuante, separados por espaços, representando as coordenadas *x*, *y* e *z* de um vértice.

Considerando que cada triângulo é definido por três vértices consecutivos no ficheiro, a leitura sequencial permite ao **engine** carregar os modelos de forma direta, sem necessidade de processar informação. Este formato minimalista reduz a complexidade tanto na escrita como na leitura dos ficheiros, tornando o sistema mais robusto e eficiente.

3. Primitivas

3.1. Plano

A geração do plano começa por dividir uma área quadrada de lado $length$, numa grelha com $divisions \times divisions$ quadrados menores, cada um com dimensão $divlength = \frac{length}{divisions}$. O plano é centrado na origem, estendendo-se de $-\frac{length}{2}$ a $\frac{length}{2}$ em ambas as direções x e z , mantendo-se sempre no plano $y = 0$ (plano XZ). Esta subdivisão será essencial para permitir um maior detalhe quando forem aplicadas transformações ou iluminação ao modelo.

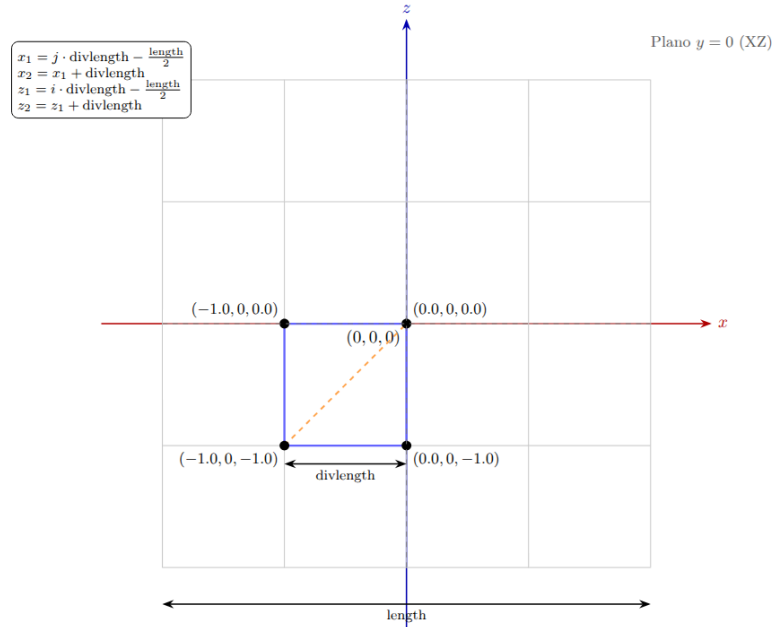


Figura 1: Diagrama para a criação da grelha do plano

Para cada quadrado da subdivisão, são definidos quatro vértices nos cantos:

$$(x_1, 0, z_1) \quad (x_2, 0, z_1) \quad (x_1, 0, z_2) \quad (x_2, 0, z_2)$$

onde

$$x_1 = j \cdot divlength - \frac{length}{2}$$

$$x_2 = x_1 + divlength$$

$$z_1 = i \cdot divlength - \frac{length}{2}$$

$$z_2 = z_1 + divlength$$

com i e j a representarem os índices da subdivisão. Cada quadrado é então decomposto em dois triângulos: o primeiro ligando os vértices $(x_1, 0, z_1)$, $(x_2, 0, z_2)$ e $(x_2, 0, z_1)$, e

o segundo ligando $(x_1, 0, z_1)$, $(x_1, 0, z_2)$ e $(x_2, 0, z_2)$. A ordem dos vértices é cuidadosamente escolhida para garantir que as normais apontam na direção positiva de y , seguindo a regra da mão direita.

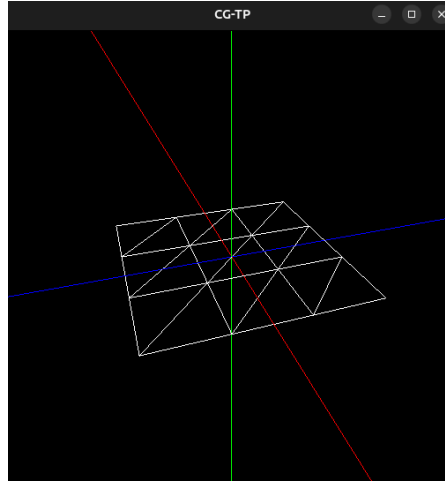


Figura 2: Resultado da geração e renderização do plano

3.2. Cubo

A construção do cubo envolve a geração de seis faces planas, cada uma subdividida numa grelha com $divisions \times divisions$ quadrados, de forma análoga ao plano. A caixa tem dimensão $length$ em todas as direções e está centrada na origem, com cada face a estender-se de $-\frac{length}{2}$ a $\frac{length}{2}$ nos seus eixos correspondentes.

As seis faces correspondem aos três pares de planos paralelos definidos pelos eixos coordenados: dois no plano XZ (topo e base), dois no plano XY (frente e trás) e dois no plano YZ (esquerda e direita).

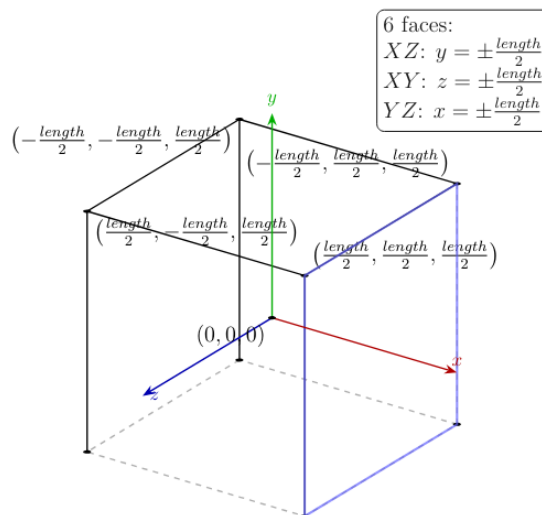


Figura 3: Diagrama para a construção do cubo

Para as faces no plano XZ , mantém-se y constante ($y = \pm \frac{length}{2}$ para o topo e base, respetivamente), variando x e z de acordo com as subdivisões. Cada quadrado nestas

faces é dividido em dois triângulos, sendo crucial inverter a ordem dos vértices na face inferior para manter as normais a apontar para o exterior. O mesmo processo é replicado para as faces nos planos XY e YZ , fixando z e x respectivamente, e variando os outros dois eixos.

A implementação garante que todas as normais apontam para fora da caixa através da ordenação correta dos vértices de cada triângulo. Por exemplo, para a face superior ($y = \frac{\text{length}}{2}$), os triângulos são definidos no sentido anti-horário quando vistos de cima, enquanto para a face inferior, a ordem é invertida. Esta atenção à orientação das faces é fundamental para o correto funcionamento do *culling*.

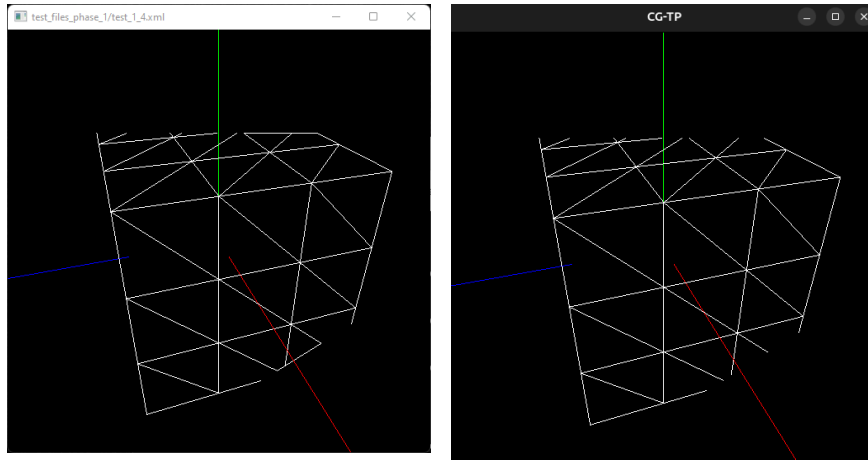


Figura 4: Comparação do cubo esperado (esquerda) com o cubo obtido (direita)

3.3. Esfera

A esfera é gerada utilizando coordenadas esféricas, definidas pelos ângulos α (azimute) e β (elevação), onde $\alpha \in [0, 2\pi]$ e $\beta \in [-\frac{\pi}{2}, \frac{\pi}{2}]$. A superfície é dividida em *slices* (fatias verticais) e *stacks* (camadas horizontais), com incrementos angulares:

$$\Delta\alpha = \frac{2\pi}{\text{slices}} \text{ e } \Delta\beta = \frac{\pi}{\text{stacks}}$$

Cada ponto na superfície da esfera com raio r é calculado através das equações paramétricas:

$$x = r \cdot \cos(\beta) \cdot \sin(\alpha)$$

$$y = r \cdot \sin(\beta)$$

$$z = r \cdot \cos(\beta) \cdot \cos(\alpha).$$

A implementação trata separadamente três regiões da esfera: o polo superior, a região intermediária e o polo inferior. No polo superior ($\beta = \frac{\pi}{2}$), são gerados triângulos que conectam o vértice do topo $(0, r, 0)$ aos vértices da primeira camada horizontal. Cada triângulo é formado pelo polo, um ponto P_1 na primeira camada no ângulo α , e o ponto adjacente P_2 no ângulo $\alpha + \Delta\alpha$. Esta abordagem evita a degeneração dos quadriláteros nos polos.

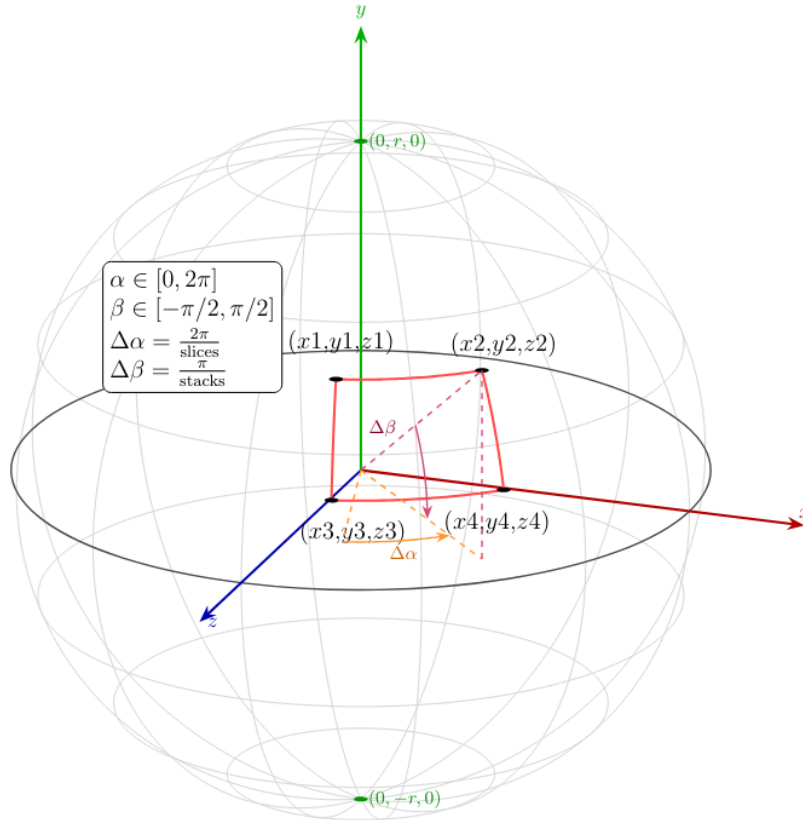


Figura 5: Diagrama para a construção da esfera com quadriláteros

Para as camadas intermédias, são criados quadriláteros entre camadas consecutivas, decompostos em dois triângulos. Dados dois níveis de elevação β_i e β_{i+1} , e dois ângulos consecutivos α_j e α_{j+1} , calculam-se quatro vértices que formam um quadrilátero. Este é então dividido nos triângulos (P_1, P_3, P_2) e (P_2, P_3, P_4) , onde P_1 e P_2 pertencem à camada superior e P_3 e P_4 à inferior. O polo inferior ($\beta = -\frac{\pi}{2}$) é tratado de forma análoga ao superior, mas com a ordem dos vértices invertida para manter a coerência das normais. Neste polo, os triângulos conectam-se ao vértice de baixo $(0, -r, 0)$.

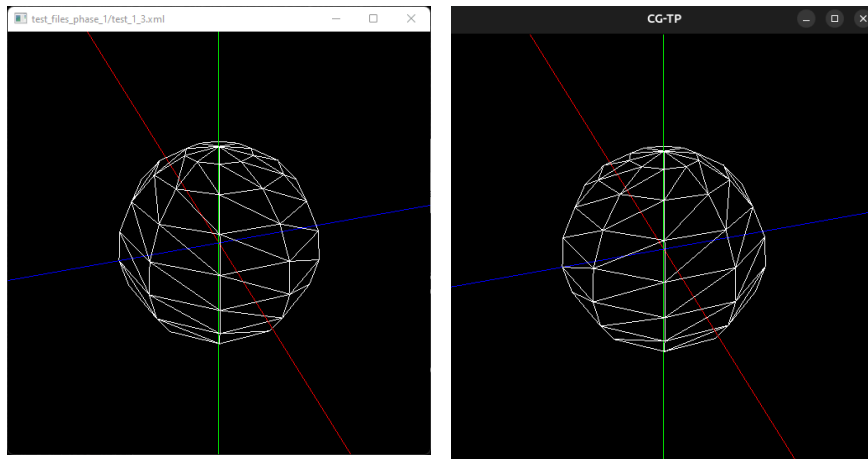


Figura 6: Comparação da esfera esperada (esquerda) com a esfera obtida (direita)

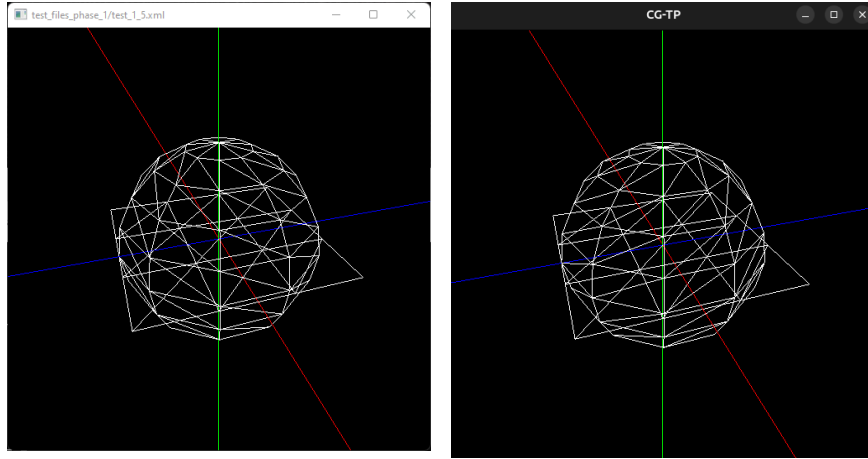


Figura 7: Comparação do output esperado com a esfera e o plano (esquerda) com o obtido (direita)

3.4. Cone

A construção do cone inicia-se com a geração da base circular no plano $y = 0$, seguida da superfície lateral que converge para o vértice no ponto $(0, h, 0)$, onde h representa a altura do cone.

A base é formada por *slices*, triângulos que partem do centro $(0, 0, 0)$ e que se estendem até à circunferência de raio r . Cada triângulo na base liga o centro a dois pontos consecutivos na circunferência, definidos por $x = r \cdot \cos(\alpha)$ e $z = r \cdot \sin(\alpha)$, onde $\alpha = i \cdot \frac{2\pi}{\text{slices}}$ com $i \in \{0, \dots, \text{slices} - 1\}$.

A superfície lateral do cone é subdividida em *stacks* (camadas horizontais), criando uma estrutura de círculos concêntricos com raios decrescentes à medida que a altura aumenta. O raio a uma altura $y = h_i$ é calculado por:

$$r_i = r \cdot \frac{h - h_i}{h}, \text{ onde } h_i = i \cdot \frac{h}{\text{stacks}}$$

Esta relação linear garante que o cone se estreita uniformemente desde a base até ao vértice. Entre duas camadas consecutivas, são formados quadriláteros que são decompostos em dois triângulos.

Para cada par de camadas em alturas y_i e y_{i+1} com raios correspondentes r_i e r_{i+1} , e para cada par de ângulos α_j e α_{j+1} , calculam-se quatro pontos: dois na camada inferior (x_1, y_i, z_1) e (x_2, y_i, z_2) , e dois na camada superior (x_3, y_{i+1}, z_3) e (x_4, y_{i+1}, z_4) . Estes formam um quadrilátero que é triangulado em (x_1, y_i, z_1) , (x_3, y_{i+1}, z_3) , (x_4, y_{i+1}, z_4) e (x_1, y_i, z_1) , (x_4, y_{i+1}, z_4) , (x_2, y_i, z_2) , garantindo a correta orientação das normais para o exterior da superfície.

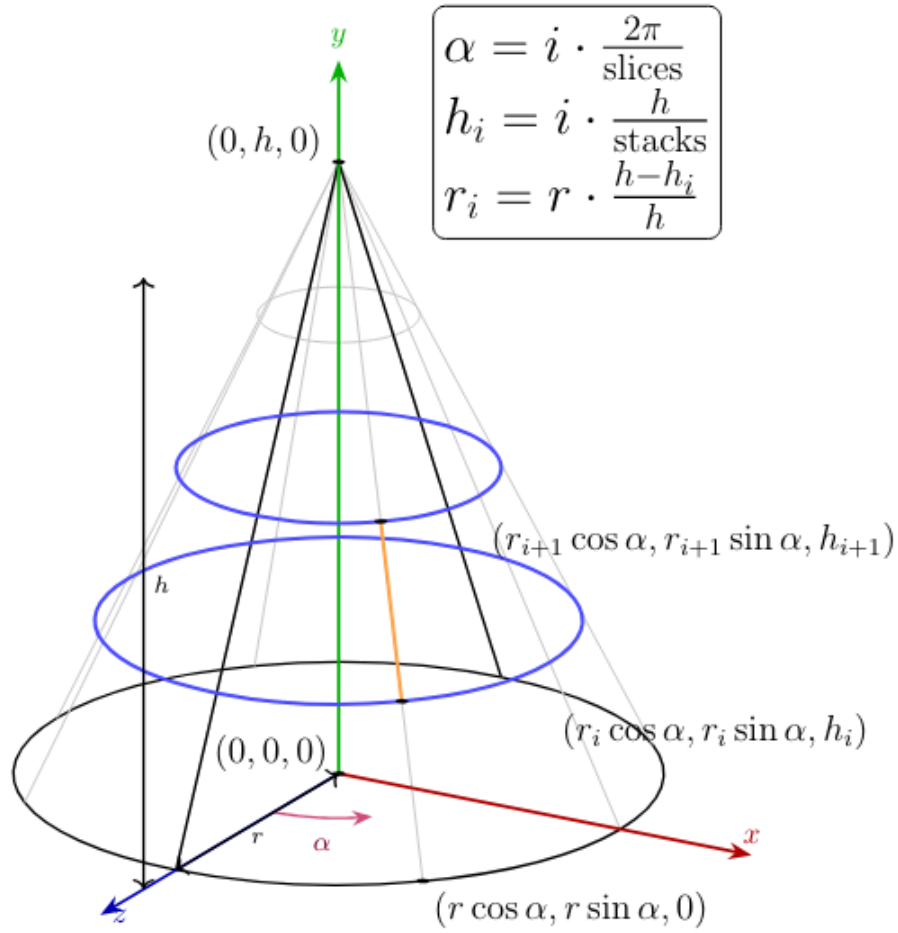


Figura 8: Diagrama para a construção do cone com stacks

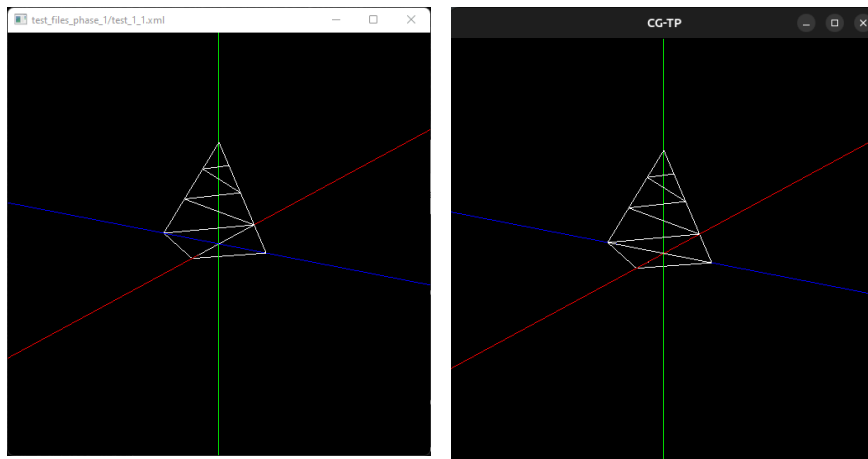


Figura 9: Comparação do cone esperado (esquerda) com o cone obtido (direita)

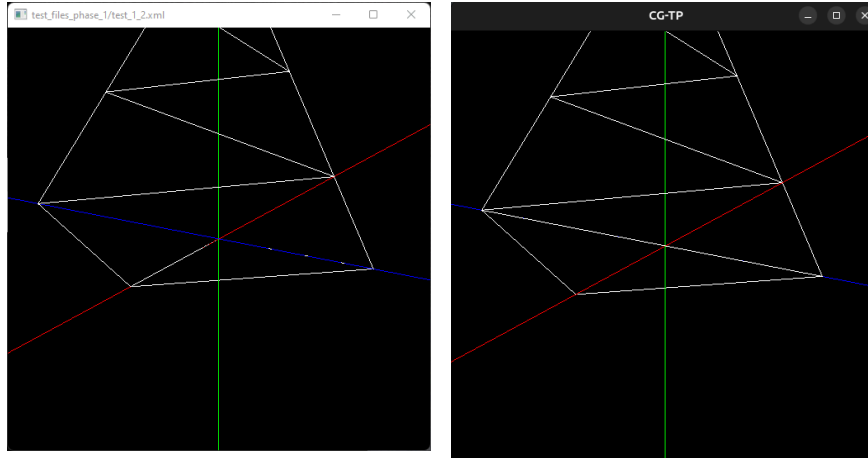


Figura 10: Outra comparação do cone esperado (esquerda) com o cone obtido (direita)

3.5. Torus

O torus é uma superfície gerada pela rotação de um círculo menor (de raio r_{minor}) em torno de um eixo que se encontra a uma distância r_{major} do centro desse círculo.

A parametrização matemática do toro utiliza dois ângulos: $\theta \in [0, 2\pi]$ que controla a rotação ao redor do eixo principal (vertical, eixo do y), e $\phi \in [0, 2\pi]$ que define a posição no círculo menor. Qualquer ponto na superfície do toro é dado pelas equações:

$$x = (r_{\text{major}} + r_{\text{minor}} \cdot \cos(\phi)) \cdot \cos(\theta)$$

$$y = r_{\text{minor}} \cdot \sin(\phi)$$

$$z = (r_{\text{major}} + r_{\text{minor}} \cdot \cos(\phi)) \cdot \sin(\theta)$$

$\theta \in [0, 2\pi]$ (rotação principal)
 $\phi \in [0, 2\pi]$ (tubo)
 $\Delta\theta = \frac{2\pi}{\text{slices}}$
 $\Delta\phi = \frac{2\pi}{\text{stacks}}$
 $R = r_{\text{major}}, r = r_{\text{minor}}$

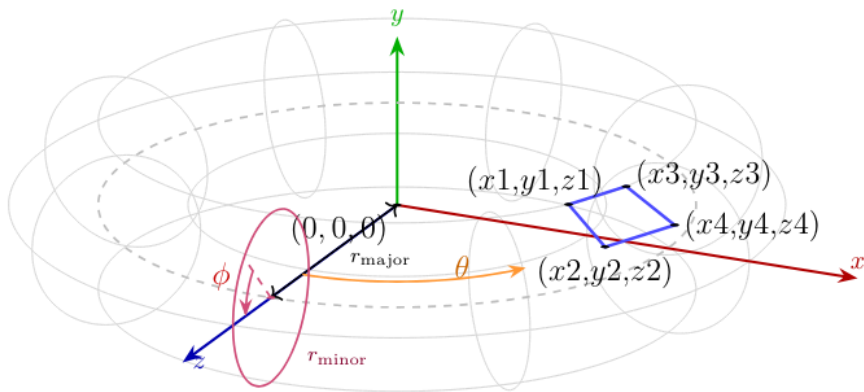


Figura 11: Diagrama para a construção do torus com slices e stacks

O desdobramento da superfície é realizado dividindo ambos os ângulos em incrementos regulares:

$$\Delta\theta = \frac{2\pi}{\text{slices}} \text{ para as fatias ao longo do círculo maior e}$$

$$\Delta\varphi = \frac{2\pi}{\text{stacks}} \text{ para as divisões ao longo do tubo}$$

Esta subdivisão cria uma malha de quadriláteros sobre a superfície do toro, onde cada quadrilátero é definido por quatro vértices correspondentes a combinações de ângulos consecutivos θ_i , θ_{i+1} , φ_j e φ_{j+1} .

Cada quadrilátero é decomposto em dois triângulos seguindo um padrão consistente. Para os vértices P_1 , P_2 , P_3 e P_4 , onde P_1 corresponde a (θ_i, φ_j) , P_2 a $(\theta_i, \varphi_{j+1})$, P_3 a $(\theta_{i+1}, \varphi_j)$ e P_4 a $(\theta_{i+1}, \varphi_{j+1})$, formam-se os triângulos (P_1, P_2, P_3) e (P_3, P_2, P_4) .

A natureza fechada do toro, sem descontinuidades ou pontos singulares, permite uma implementação elegante onde todos os quadriláteros são tratados uniformemente, sem necessidade de casos especiais como acontece nos polos da esfera.

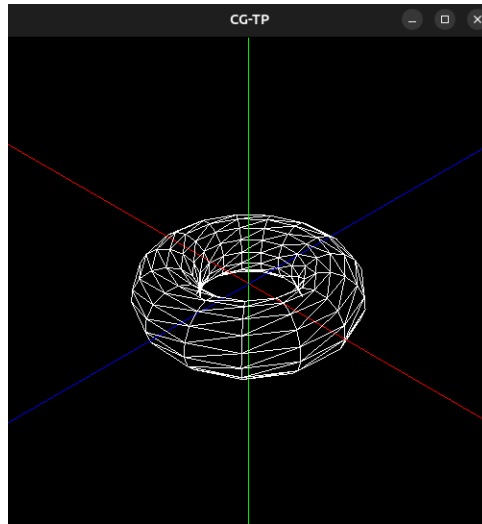


Figura 12: Resultado da geração e renderização do torus

3.6. Cilindro

O cilindro é composto por três componentes distintas: duas bases circulares (superior e inferior) e a superfície lateral que as conecta.

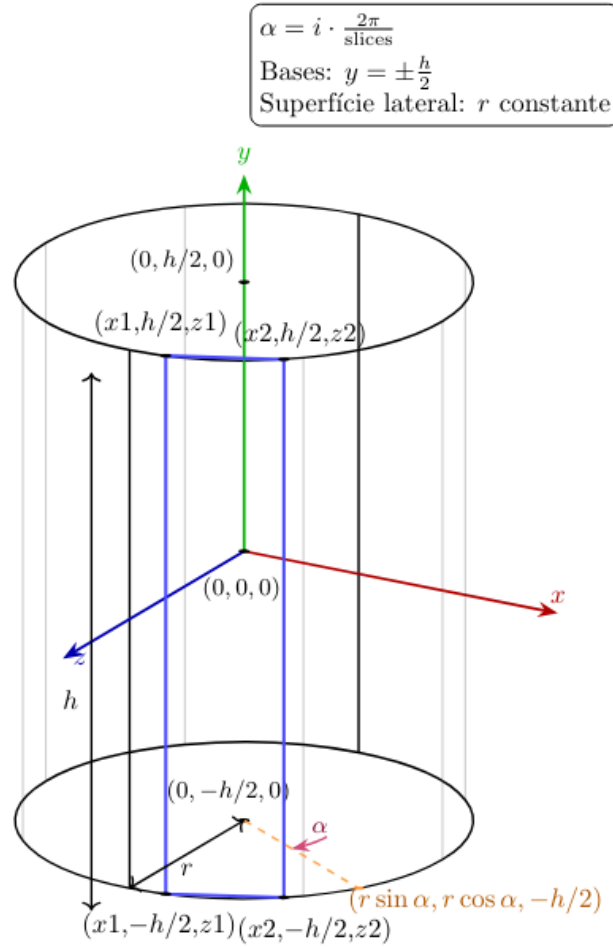
As bases estão posicionadas em $y = \pm \frac{h}{2}$, onde h é a altura total do cilindro, e têm raio r . A geração começa pelas bases, onde cada uma é formada por *slices*, triângulos que partem do centro da circunferência e se estendem até pontos na sua periferia, calculados através de $x = r \cdot \sin(\alpha)$ e $z = r \cdot \cos(\alpha)$, com $\alpha = i \cdot \frac{2\pi}{\text{slices}}$ para $i \in \{0, \dots, \text{slices} - 1\}$.

Na base inferior ($y = -\frac{h}{2}$), cada triângulo é definido ligando o centro $(0, -\frac{h}{2}, 0)$ a dois pontos consecutivos na circunferência, com a ordem dos vértices $(0, -\frac{h}{2}, 0)$, $(x_2, -\frac{h}{2}, z_2)$, $(x_1, -\frac{h}{2}, z_1)$ garantindo que a normal aponte para baixo.

De forma simétrica, a base superior ($y = \frac{h}{2}$) utiliza a ordem $(x_2, \frac{h}{2}, z_2)$, $(0, \frac{h}{2}, 0)$, $(x_1, \frac{h}{2}, z_1)$ para que a normal aponte para cima.

Esta inversão na ordenação dos vértices é crucial para manter a consistência das normais exteriores.

A superfície lateral é construída através de *slices* quadriláteros verticais, cada um ligando dois pontos consecutivos da base inferior aos correspondentes pontos na base superior. Para cada par de ângulos α_i e α_{i+1} , obtêm-se quatro vértices: $(x_1, -\frac{h}{2}, z_1)$, $(x_2, -\frac{h}{2}, z_2)$, $(x_1, \frac{h}{2}, z_1)$ e $(x_2, \frac{h}{2}, z_2)$.



Cada quadrilátero é dividido em dois triângulos com os respectivos pontos:

$$(x_2, \frac{h}{2}, z_2), (x_1, \frac{h}{2}, z_1), (x_1, -\frac{h}{2}, z_1)$$

$$(x_2, -\frac{h}{2}, z_2), (x_2, \frac{h}{2}, z_2), (x_1, -\frac{h}{2}, z_1)$$

Esta decomposição garante que as normais apontam para o exterior do cilindro.

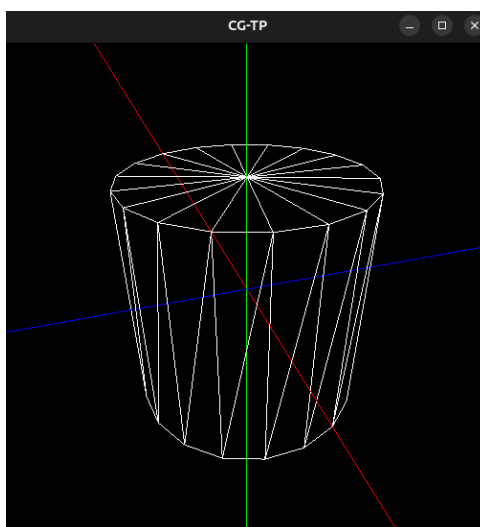


Figura 14: Resultado da geração e renderização do cilindro

4. Engine

4.1. Função

O **engine** é o componente central do sistema de visualização 3D, responsável por interpretar ficheiros XML que descrevem cenas 3D e renderiza-las utilizando o OpenGL. A sua arquitetura baseia-se numa *pipeline* de processamento que se inicia com o *parsing* do ficheiro XML de configuração, onde são extraídos os parâmetros da câmara (posição, ponto de foco (*look at*), vetor *up* e parâmetros de perspetiva), as dimensões da janela de visualização, e as referências aos ficheiros de modelos 3D que compõem a cena.

Esta separação entre a descrição da cena e os dados geométricos dos modelos permite uma grande flexibilidade na composição de diferentes cenários sem necessidade de regenerar os modelos.

O processamento do XML é realizado através da biblioteca TinyXML2. A função `processXML` percorre esta estrutura hierárquica, extraíndo primeiro as configurações globais da janela, seguidas dos parâmetros da câmara organizados em elementos `<position>`, `<lookAt>`, `<up>` e `<projection>`.

Após esta fase de configuração, a função processa o elemento `<group>` que contém a lista de modelos através de elementos `<model>`, cada um especificando um ficheiro `.3d` a ser carregado. Os nomes destes ficheiros são armazenados num vetor global que será posteriormente utilizado na fase de renderização.

A renderização propriamente dita ocorre na função `renderScene`, invocada pelo ciclo principal do GLUT sempre que a janela necessita de ser redesenhada. Esta função começa por limpar os *buffers* de cor e profundidade, configura a matriz de visualização utilizando `gluLookAt` com os parâmetros da câmara previamente carregados, e desenha os eixos coordenados como referência visual. De seguida, itera sobre todos os ficheiros de modelo carregados, invocando para cada um a função `createModel` que lê o ficheiro `.3d`, extrai os vértices linha a linha, e submete-os ao OpenGL através de

chamadas `glVertex3f` dentro de um bloco `GL_TRIANGLES`. O motor está configurado para renderizar em modo *wireframe* através de `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)`, permitindo visualizar a estrutura triangular dos modelos.

4.2. Teste externo

Para validar a correta implementação do **engine** de forma mais abrangente, para além dos testes realizados com os modelos gerados pelo **generator**, optámos também por testar a renderização de um modelo externo obtido através da *Internet*.

Selecionamos um modelo 3D de uma vaca que, após ser convertido para o formato `.3d` utilizado pelo nosso sistema, permitiu verificar a capacidade do motor gráfico em processar geometrias não geradas internamente.

Este teste demonstrou que o **engine** é suficientemente robusto e genérico para carregar e renderizar qualquer modelo que respeite o formato `.3d` estabelecido, independentemente da sua origem ou complexidade geométrica, comprovando assim a flexibilidade e a correta implementação da nossa solução.

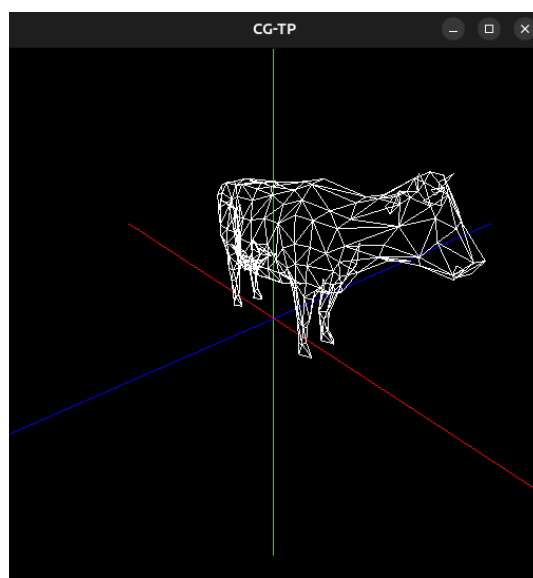


Figura 15: Renderização de um modelo 3D de uma vaca obtido na Internet

5. Conclusão

Nesta primeira fase do projeto, foram concretizados com sucesso os objetivos propostos no enunciado, nomeadamente a implementação do **generator** capaz de criar modelos das quatro primitivas geométricas requeridas (plano, cubo, esfera e cone) e do **engine** com capacidade para interpretar ficheiros XML e renderizar as cenas descritas. A arquitetura modular adotada em ambos os programas mostrou-se eficaz, permitindo uma clara separação entre a lógica de geração de modelos e a de renderização, facilitando a manutenção e compreensão do código. O formato `.3d` escolhido para o armazenamento dos modelos revelou-se adequado, oferecendo simplicidade sem comprometer a funcionalidade necessária.

Para além dos requisitos mínimos, foram implementadas funcionalidades adicionais que enriqueceram o projeto. A criação de primitivas extra, como o torus e o cilindro, permitiu explorar diferentes abordagens de definição de superfícies e demonstrou a flexibilidade do sistema desenvolvido. O teste com o modelo externo da vaca foi particularmente importante, pois validou a capacidade do *engine* de processar geometrias arbitrárias que respeitem o formato *.3d*, indo além dos modelos gerados internamente e comprovando a robustez da implementação.

Apesar dos resultados satisfatórios obtidos, a solução atual apresenta algumas limitações que serão abordadas nas fases seguintes do projeto. Nomeadamente, a câmara encontra-se totalmente estática, sendo configurada apenas através do ficheiro XML sem possibilidade de interação em tempo real. A implementação de uma câmara interativa, permitirá ao utilizador navegar pela cena através do teclado e rato, constituindo uma evolução fundamental para as próximas fases.

6. Bibliografia

Free3D. *Low Poly Cow 3D Model*. Disponível em: <https://free3d.com/3d-model/low-poly-cow-75171.html>.

Weisstein, Eric W. *Torus*. MathWorld—A Wolfram Web Resource. Disponível em: <https://mathworld.wolfram.com/Torus.html>.